Local AI Coding Agent in VS Code: Complete Setup Guid

Recommended stack: Cline (VS Code extension) + qwen/qwen3-30b-a3b-thinking-2507 (Q4_K_M)
Inference backend: LM Studio Bionic Local Model API at http://localhost:1235/v1 (llama.cpp, Vulkan)

Environment: Windows 11 (no WSL2) | AMD Radeon RX 9060 XT, 16 GB VRAM | Local git + LaTeX Workshop | October 29, 2026 | Pre

Overview of the Recommended Stack

Component	Choice	Rationale
VS Code extension	Cline	The only maintained open-source VS Code agent with a dedicated LM Studio provider, a compact-prompt mode for local models, a visible Thinking section, and granular auto-approval up to full YOLO mode. Roo Code was archived in May 2026; Kilo Code works but adds multi-agent features a beginner does not need.
Model	qwen/qwen3-30b-a3b-thinking-2507	Always-thinking reasoning model explicitly flagged "Trained for tool use" in the LM Studio catalog; 30.5B total parameters with only 3.3B active per token, so it runs acceptably even when part of the weights sit in system RAM.
Quantization	Q4_K_M GGUF (approx. 18.6 GB)	The practical 4-bit floor recommended for coding agents; Q3 quants measurably degrade tool-call reliability. On 16 GB VRAM a small share of the experts is held in system RAM; LM Studio lists 17 GB minimum system memory.
Endpoint	http://localhost:1235/v1	Your existing LM Studio Bionic Local Model API with just-in-time model loading; the Vulkan backend of llama.cpp serves the Radeon GPU.
Permissions	Fully automatic (YOLO Mode)	Per your preference; safety net provided by Cline checkpoints plus the local git repository (see Sections 4 and 5).

Important: Why not gpt-oss-20b? Although it fits fully within 16 GB VRAM and is a strong native tool caller, it is unusable with Cline: Cline drives models with its own XML tool-call format rather than native function calling, and gpt-oss answers in its native harmony format. The Cline team confirmed gpt-oss models "are terrible in Cline", and AMD's Cline + LM Studio model testing excluded gpt-oss-20b for broken tool behavior. Qwen3-30B-A3B-Thinking-2507 is the model that satisfies all three of your requirements (thinking, tool calling, local git commits) at this VRAM tier.

Prerequisites Checklist

- Windows 11; AMD Radeon RX 9060 XT (16 GB VRAM); at least 32 GB system RAM recommended (the model spills 2-3 GB into system RAM at Q4_K_M).
- LM Studio Bionic installed, Local Model API running at http://localhost:1235/v1, just-in-time model loading enabled (as per your setup).
- VS Code with LaTeX Workshop and a local git repository at D:\E\Wissensbasis\Projekte\Cheatsheet_collection.
- pdflatex available on the Windows PATH (MiKTeX or TeX Live); verify with pdflatex --version in PowerShell.
- Approximately 19 GB free disk space for the model download.

Step 1: Configure LM Studio Bionic

1.1 Download the model

1. In Bionic, open Settings -> Local Models -> Explore (in classic LM Studio this tab is called Discover).
2. Search for "qwen3-30b-a3b-thinking-2507" and select the entry qwen/qwen3-30b-a3b-thinking-2507 (GGUF, publisher lmstudio-community).
3. Choose the Q4_K_M quantization (approximately 18.6 GB) and confirm the download. This is the only model you need.
4. Ignore any device-fit warning that the file exceeds 16 GB VRAM: only 3.3B parameters are active per token, so a partial hold in system RAM costs little speed.

1.2 Set the model default load configuration (critical)

Known issue: just-in-time loading defaults to a 16,384-token context window, while Cline's system prompt including tool definitions already consumes roughly 25,000 tokens. With the default you will see "context window exceeded" errors or a model that loses the task after two or three tool calls. Fix this once, permanently:

1. Open My Models (sidebar) and find the card for qwen/qwen3-30b-a3b-thinking-2507.
2. Click the gear icon on the model card to open the default load parameters.
3. Set Context Length to 32768 and GPU offload to the maximum suggested value, then save. These parameters apply to every future JIT load.

Alternative via the lms command line (Bionic 1.1.8 restored lms compatibility):

```
lms unload qwen/qwen3-30b-a3b-thinking-2507
lms load qwen/qwen3-30b-a3b-thinking-2507 --context-length 32768
lms ps
```

Leave KV cache quantization disabled: it can persist context between tasks and produce unpredictable agent behavior; Cline's own LM Studio guide recommends keeping it off.

1.3 Verify the server settings

Setting (Server Settings / Developer)	Value	Reason
Server Port	1235	Your Bionic setup; Cline default is 1234 and must be corrected in Step 3.
Require Authentication	OFF	Cline's dedicated LM Studio provider has no API-key field (known gap, cline issue #9668). If you need authentication later, use Cline's "OpenAI Compatible" provider instead.
Serve on Local Network	OFF	Nothing outside this machine should reach the endpoint.
Enable CORS	OFF	CORS is only required by browser-based clients (for example Open WebUI); the VS Code extension does not need it.
Just in Time Model Loading	ON	Keeps your existing setup; the load defaults from 1.2 apply.
Auto Unload Unused JIT Models	your choice	ON saves VRAM between sessions; OFF keeps the model resident and avoids reload times.

1.4 Enable reasoning separation

In the Developer settings, enable "When applicable, separate reasoning_content and content in API responses". This returns the model's chain of thought in the dedicated reasoning field of the OpenAI-compatible API, which Cline renders as the collapsible Thinking section instead of mixing reasoning

into the visible answer.

1.5 Verify the endpoint

In PowerShell, run the following and confirm the model id appears:

```
curl http://localhost:1235/v1/models

Expected response (excerpt):
{ "data": [ { "id": "qwen/qwen3-30b-a3b-thinking-2507", "object": "model" } ] }
```

Note: Copy the exact "id" string. If it deviates from the name above (for example it includes a quantization suffix), use the exact string in Cline in Step 3.

Step 2: Install the Cline Extension in VS Code

1. Open VS Code, then Extensions (Ctrl+Shift+X).
2. Search for "Cline"; install the extension by publisher Cline Bot Inc. (5M+ installs, Apache 2.0). Reload VS Code when prompted.
3. Open your project: File -> Open Folder -> D:\E\Wissensbasis\Projekte\Cheatsheet_collection.
4. The Cline icon appears in the Activity Bar on the left; click it to open the Cline panel.

Step 3: Connect Cline to the Bionic Endpoint

1. In the Cline panel, open Settings via the gear icon.
2. API Provider: select LM Studio.
3. Base URL: change the default http://localhost:1234 to http://localhost:1235/v1.
4. Model: select qwen/qwen3-30b-a3b-thinking-2507 from the model dropdown (the list is read from your endpoint; the first request triggers the JIT load, which takes 20-60 seconds).
5. Stay in Cline Settings, open the Features section, and enable "Use Compact Prompt".

Why this matters: "Use Compact Prompt" reduces Cline's system prompt to roughly one tenth of its full size, which is decisive for a local model with a 32k context. The trade-off disables MCP tools, Focus Chain and MTP features; none of these are needed for editing .tex files, compiling and committing.

Fallback: if you ever enable "Require Authentication" in Bionic, switch the provider in Cline from LM Studio to "OpenAI Compatible", set Base URL to http://localhost:1235/v1, enter the Bionic API token as the API key, and paste the exact model id from Step 1.5 as the Model ID.

Step 4: Permissions for Fully Automatic Operation

4.1 Granular option: Auto Approve

In the Cline settings panel, locate the Auto Approve section and enable the following toggles:

- Read project files: ON
- Edit project files: ON
- Execute safe commands: ON
- Execute all commands: ON

The distinction matters: git status, ls or cat count as safe commands, while pdflatex runs and git commit are classified as approval-required commands. "Execute all commands" is therefore the toggle that makes compiling and committing fully automatic.

4.2 Single-switch option: YOLO Mode

Alternatively, in Cline Settings -> Features, check "YOLO Mode". This auto-approves every action: all file operations, all terminal commands including destructive ones, browser actions, MCP tools and mode transitions. This matches your stated preference for fully automatic operation; combine it with the safety measures below.

4.3 Safety measures for unattended operation

- The local git repository is the primary safety net: every committed state can be restored (git log, git checkout).
- Cline creates a checkpoint at every step; use the Compare and Restore buttons in the task header to roll back any change.
- Section 5 adds a project rule that forbids pushing to remotes, so automation stays local.
- Watch the first few sessions completely before leaving tasks unattended.

Step 5: Project Rules for the Cheatsheet Repository

Create the file D:\E\Wissensbasis\Projekte\Cheatsheet_collection\.clinerules with the following content. Cline reads it automatically at the start of each task:

```
This repository contains LaTeX cheatsheets compiled with pdflatex.

- After each completed change to a .tex file, compile it with pdflatex and fix any
LaTeX errors before finishing the task.
- After a successful task, stage the changed files with git add and create a local
git commit with a concise, descriptive message. Never push to any remote.
- Keep edits minimal; preserve the existing LaTeX preamble and formatting conventions.
```

Rules can be toggled per session in the rules popover below the Cline chat input.

Step 6: Verification of the Three Requirements

Test 1: Thinking (reasoning mode)

Prompt in the Cline chat: "Think step by step: what is the sum of the integers from 1 to 100, and why?"

- Pass criterion: a collapsed "Thinking" section appears above the answer; click it to inspect the chain of thought.
- If the reasoning text appears inside the answer instead, re-check Step 1.4 (reasoning separation).

Test 2: Terminal execution (compile a cheatsheet)

Prompt: "Compile <your-sheet>.tex with pdflatex and report whether the PDF was produced and how many pages it has." (Replace <your-sheet> with an actual file name in the repository.)

- Pass criterion: Cline runs pdflatex in the VS Code terminal without any approval dialog, reads the output, and reports the result; the corresponding .pdf file exists in the repository folder.
- The first run includes the JIT model load (20-60 seconds). Subsequent runs start immediately.

Test 3: Automatic git commit

Prompt: "Stage the current changes and commit them with a descriptive message."

- Pass criterion: no approval prompt appears; afterwards run in the VS Code terminal:
- git log --oneline -3 -> the new commit is at the top
- git status -> working tree clean (or only intended untracked files)
- The commit is local only; nothing is pushed (see Section 5).

Context check

While Cline works, watch the token bar at the top of the task view: total usage should stay clearly below 32,768 tokens. If it approaches the limit, start a new task (plus button in the task header) rather than letting the context overflow.

Step 7: Troubleshooting Known Issues

Symptom	Cause	Fix
"Context window exceeded" or "Cannot truncate prompt" errors; model forgets the task after 2-3 tool calls.	JIT default context of 16,384 tokens; Cline needs about 25k for its system prompt.	Set Context Length 32768 in the model default load parameters (Step 1.2); confirm "Use Compact Prompt" is on.
HTTP 500 error on every Cline request.	Model ID mismatch between Cline and the endpoint.	Copy the exact id from http://localhost:1235/v1/models into Cline.
Cline stuck at "API Request...".	Server not running or model not loaded.	Check the server status in the Bionic Developer panel; re-run curl http://localhost:1235/v1/models.
Thinking text mixed into the normal answer.	Reasoning separation disabled.	Enable the separate reasoning_content setting (Step 1.4).
Generation is very slow.	Expert weights partially held in system RAM; low free system RAM.	Close memory-intensive applications; reduce Context Length to 24,576; as a last resort use a Q4_K_S or Q3_K_L quant (fully GPU-resident, but Q3 measurably degrades tool-call reliability).
Extension cannot connect although curl works.	Wrong port in Cline.	Base URL must be http://localhost:1235/v1 (Cline defaults to 1234).
Model produces broken or missing tool calls.	Substituted model not suited to Cline (for example gpt-oss-20b, DeepSeek-R1 distills, 7B-class models).	Use qwen/qwen3-30b-a3b-thinking-2507 as recommended; Cline demands reliable structured tool calls.

Key Sources

- Cline documentation, Run Cline with local models (LM Studio provider, Use Compact Prompt): docs.cline.bot/running-models-locally/overview
- Cline documentation, Auto Approve and YOLO Mode: docs.cline.bot/features/auto-approve
- Cline blog, "Cline + LM Studio: the local coding stack" (context window, KV cache quantization off, 4-bit guidance): cline.bot/blog/local-models
- LM Studio documentation, Server Settings (Server Port, Require Authentication, Enable CORS, JIT loading): lmstudio.ai/docs/developer/core/server/settings
- LM Studio model page qwen3-30b-a3b-thinking-2507 (Trained for tool use; Reasoning; 17 GB minimum system memory): lmstudio.ai/models/qwen/qwen3-30b-a3b-thinking-2507
- LM Studio API changelog (separate reasoning_content setting; reasoning fields in the OpenAI-compatible API): lmstudio.ai/docs/developer/api-changelog
- Bionic changelog 1.1.8 (lms compatibility with Bionic): lmstudio.ai/changelog/bionic-v1.1.8
- aicoderscope, "LM Studio Bionic as a Coding Agent in 2026" (Cline provider base URL, >25k context for agentic tools, missing API-key field cline#9668): aicoderscope.com/blog/lm-studio-bionic-coding-agent-privacy-guide-2026
- dev.to (Jovan Chan), "Cline + LM Studio 2026: complete setup guide, the 32k context trap"
- VDF AI, "Best Local LLM for Agents and Tool Calling" (October 2026 shortlist by VRAM tier): vdf.ai/blog/best-local-llm-for-agents-tool-calling

- modelfit, "Best Local LLMs for Cline (2026 Tier List)" (tool-call reliability, Q4_K_M floor, Q3 degradation): modelfit.io/tools/cline
- Community reports on gpt-oss in Cline (r/CLine; AMD model-testing thread; Cline team statement on gpt-oss tool-call incompatibility)

Guide compiled on October 29, 2026. Tool versions move quickly; if a menu label deviates, the setting names above follow the official documentation as of this date.